Documento de arquitectura: Wikipedia Article Analysis

Sistemas distribuidos I
Cátedra: Roca

Integrante	Padrón
Franco Guardia	109374
Pilar Paz Blanco	105600
Tobias Rivero Trujillo	106302

Índice:

Scope **3**
Architecture Overview **3**
Architectural Goals & Constraints **5**
Scenarios **6**
Size and Performance **7**
DAG **8**
Diagrama de Contexto y Contenedores (C4 Model) **10**
Componente 14
Diagrama de Actividades **16**
Diagrama de Robustez **21**
Diagrama de Secuencia **24**
Diagrama de Paquetes **29**
Diagrama de Despliegue **30**
Resolución Distribuida de las 5 Queries **30**
Lista de tareas y división **31**

Scope

El sistema propuesto busca simular un sistema de queries. Habrá 5 queries conocidas y fijas, a las cuales llamaremos Q1,Q2,Q3,Q4 y Q5. El sistema se lanzará y quedará a la espera de la conexión de un cliente y el cliente por su parte será otro binario que realizará un tarea simple: enviar datos hasta terminar su fuente y luego esperar las respuestas a las queries mencionadas. Al recibir todas las respuestas el mismo finalizara. Por lo tanto no será posible:

- Enviar datos y esperar respuesta a una query en particular.
- Volver a enviar datos al terminar las 5 consultas. El cliente podrá volver a realizar una consulta pero el sistema lo tratará como un nuevo usuario.
- Esperar un orden particular de queries.
- Enviar información parcial de su fuente de datos, es decir tarde o temprano enviará todos los registros de su parquet ni más ni menos.

Por otro lado la fuente de datos será un subset de parquets provenientes del dataset provisto sobre wikipedia, por lo que se conoce el formato y tipo de data a ser enviado y se espera que dicho esquema sea respetado en la comunicación con nuestro sistema. Para el sistema final se busca poder escalar y manejar suficientemente bien los recursos del sistema distribuido para soportar y responder a la totalidad del dataset de artículos de Wikipedia con un tamaño aproximado de 50 GB para responder a las consultas correctamente. En lugar de procesar la información en un único hilo con la librería Pandas , la solución emplea un enfoque de sistemas distribuidos orientado a mensajes, utilizando Python, Docker y RabbitMQ como Middleware Orientado a Mensajes (MOM).

Architecture Overview

El sistema busca solucionar la problemática de forma distribuida. Un conjunto de procesos independientes (gateway, workers y middleware) colabora para resolver las cinco consultas sobre los artículos de Wikipedia, sin que ninguno de ellos necesite conocer el dataset completo. El sistema se presenta como una única unidad que interactúa con dos actores externos: los clientes, que envían los registros de su fuente de datos y esperan los resultados de Q1 a Q5, y la API de Wikimedia, de la cual se obtienen las imágenes requeridas por Q2.

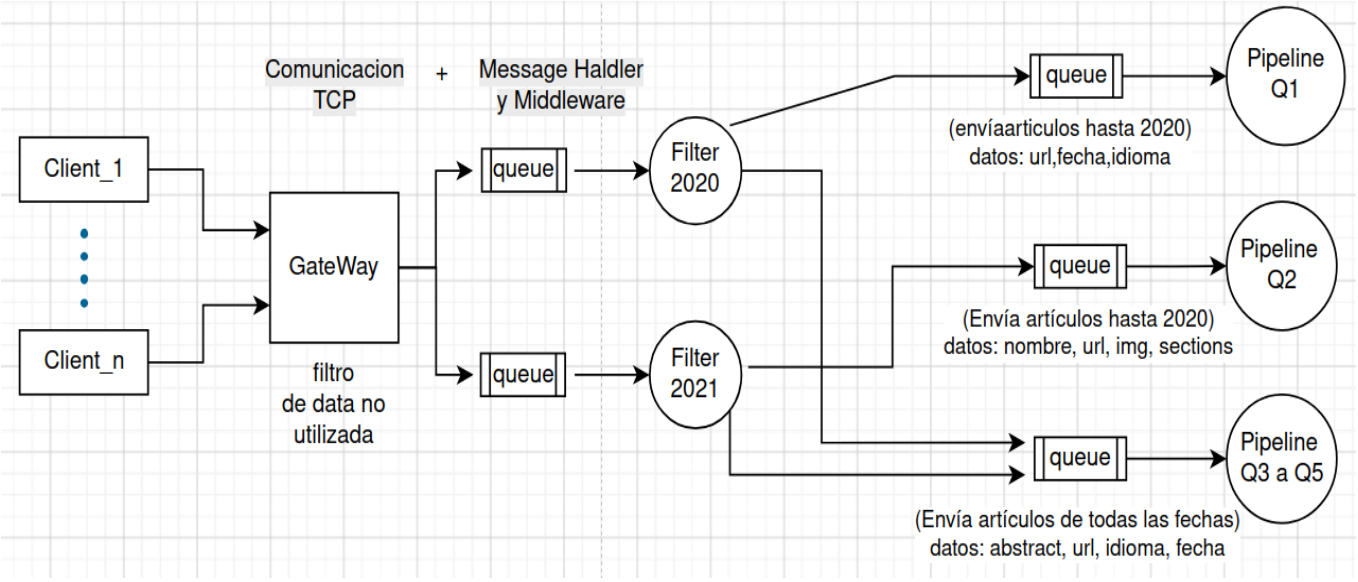


Diagrama de referencia del flujo de punta a punta. Se tienen varios clientes, y los artículos se rutean en base a las fechas requeridas para cada query.

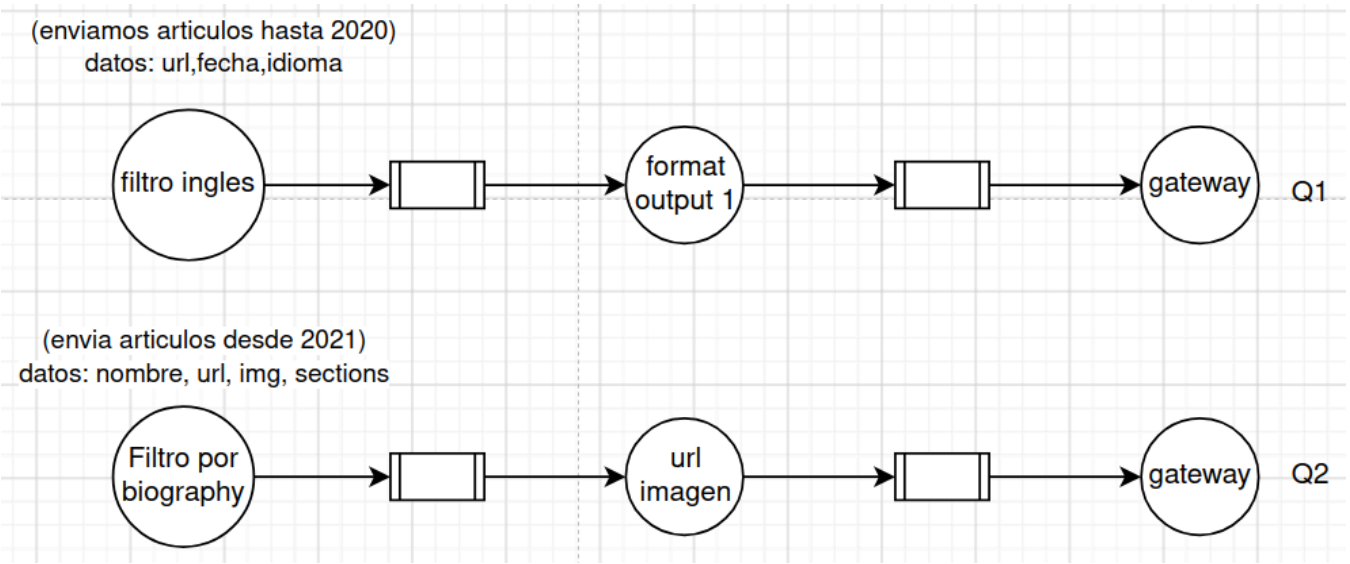
Componentes principales.

- Cliente: binario simple que lee su fuente de datos por lotes, la envía al sistema y almacena los resultados a medida que llegan. No realiza ningún procesamiento más allá del necesario para respetar el protocolo.
- Gateway y Message Handler: se conecta vía TCP a los clientes, confirma la admisión de lotes, comunica con el middleware y devuelve los resultados al cliente correspondiente.
- Middleware (RabbitMQ): abstrae la comunicación entre todos los roles, mediante exchanges y colas.
- date_router: clasifica por fecha y proyecta los campos requeridos por cada rama.
- word_ocurrences: comparte la tokenización y el filtrado de stopwords y números entre Q3, Q4 y Q5.
- Workers de consulta: implementan la lógica de cada query. Q1 y Q2 operan sobre artículos hasta 2020 inclusive; Q3, Q4 y Q5 parten de una etapa común de análisis de abstracts (stopwords y ocurrencia de palabras).

Flujo Punta a Punta:

1. **Input Eficiente:** El "Nodo Cliente" lee el archivo particionado **.parquet** en chunks parametrizable, evitando cargar el archivo completo en la memoria RAM. Inmediatamente, se descartan las columnas irrelevantes del dataset de Kaggle para no propagar datos innecesarios por la red.
2. **Procesamiento de Carga Pesada:** Los registros base se envían por RabbitMQ mediante una *Work Queue*. Un pool escalable de "Workers" recibe estos datos y ejecuta el cómputo pesado una única vez por artículo: tokenización con expresiones regulares, eliminación de *stopwords* y *numbers*, y parseo de los JSON anidados.

3. **Distribución:** Cada uno de estos Worker genera pequeños Data Transfer Objects específicos para cada query y los emite a diferentes *Exchanges* de RabbitMQ. Esto minimiza la redundancia de datos y reduce drásticamente el tamaño de la información en tránsito.
4. **Agrupación y Filtrado:**
    - Los nodos sin estado (Filtros Q1 y Q2) evalúan reglas simples (fechas e idiomas) y escalan fácilmente.



Diagramas de referencia para el flujo de las queries Q1 y Q2.

    - Los nodos con estado (Q3, Q4 y Q5) utilizan enrutamiento por *Hash* o clave directa de RabbitMQ. Esto asegura que la misma palabra siempre vaya al mismo contenedor de agregación, permitiendo el escalado horizontal sin colisiones (regla a definir).

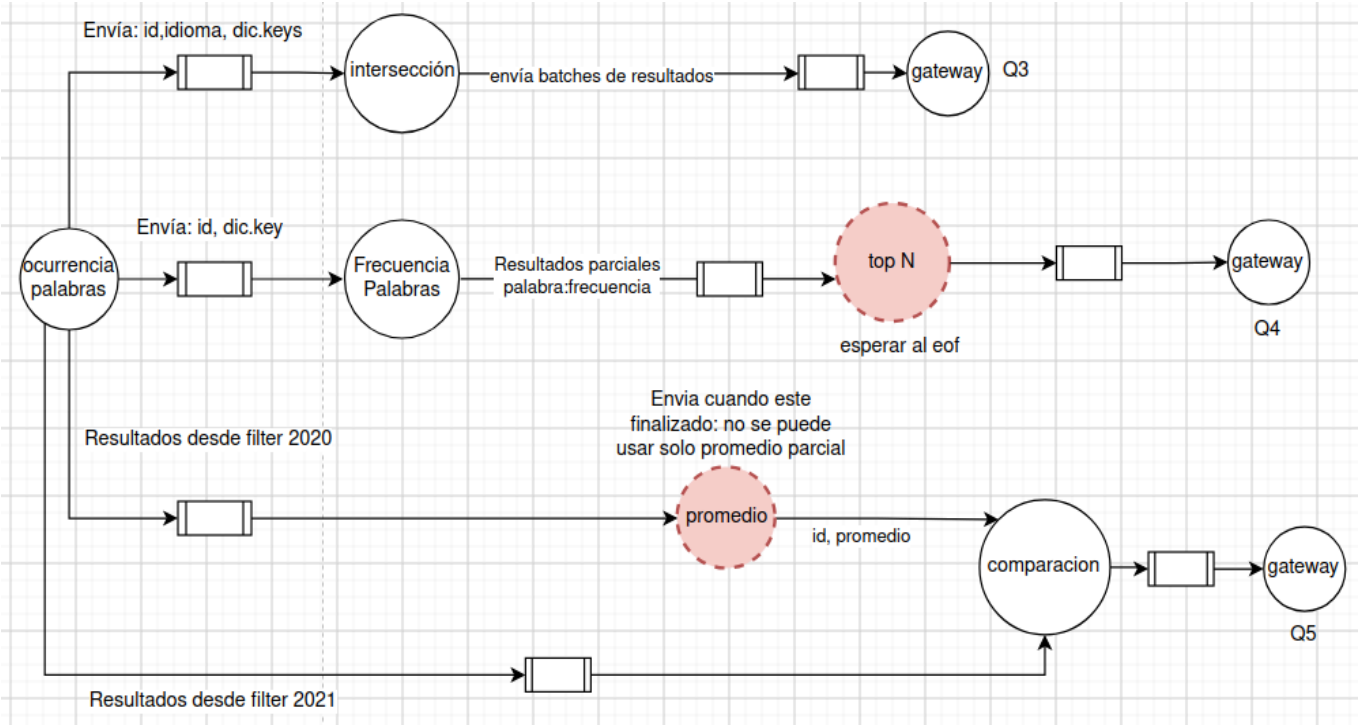


Diagrama de referencia para las queries Q3, Q4 y Q5.

5. **Sincronización por Barrera (Query 5):** La Query 5 requiere el promedio global de los artículos **<= 2020** para filtrar los artículos **> 2020**. Se implementa un nodo filtro que se encarga de direccionar para donde corresponda el paquete en cuestión.
6. **Sincronización y Cierre (EOF):** El cierre de una entrada se coordina dentro del middleware por ejecución, grupo y entrada lógica. Cada productor publica un EOF con la cantidad de lotes enviados, incluso cuando esa cantidad es cero. La coordinadora del grupo receptor reúne los cierres de todos los productores esperados y solicita a sus réplicas informar los lotes resueltos. Cuando todas informaron y la suma coincide con el total anunciado, difunde la finalización de la entrada. Cada worker ejecuta entonces su finalizador, publica los resultados pendientes y cierra sus propias salidas. Los mensajes de control utilizan colas separadas de los datos, por lo que el algoritmo contempla que un EOF llegue antes que los últimos lotes.

Architectural Goals & Constraints

Los objetivos principales de la arquitectura son distribuir el procesamiento, atender clientes concurrentes y permitir el crecimiento independiente de los distintos roles. La cantidad de instancias se establece mediante configuración, sin un máximo fijo impuesto por el programa. La topología permanece estable durante cada ejecución.

El aislamiento entre clientes debe mantenerse en los mensajes, acumuladores, archivos temporales y resultados. Para una misma entrada y configuración, el contenido de las respuestas debe ser determinista, aunque el orden de emisión pueda variar por la concurrencia.

El diseño busca evitar trabajo repetido mediante la reutilización del análisis de abstracts entre Q3, Q4 y Q5. También reduce el tráfico de red descartando tempranamente los campos innecesarios y conservando en cada rama únicamente los que requieren sus etapas posteriores.

Se establecen las siguientes restricciones:

- La implementación utiliza Python y un middleware propio basado en RabbitMQ/Pika para abstraer la comunicación por grupos.
- La escalabilidad es estática ya que las cantidades de instancias se definen en **config/config.json**, dentro de **grupos**. Un generador produce los servicios e identidades del Compose.
- El gateway, la recepción de resultados del gateway, Average y Comparator tienen una instancia. El generador admite hasta dos instancias de Image Downloader. Top N admite varias, pero cada ejecución se concentra en una de ellas.
- No se utilizan bases de datos. El estado que no pueda mantenerse en memoria acotada se administra mediante archivos propios y acceso incremental.
- El cliente procesa los archivos de entrada por lotes, sin cargar el dataset completo en memoria.
- La admisión de lotes utiliza confirmaciones entre cliente y servidor

- Hay límites en los buffers y en los mensajes pendientes de las etapas internas, para no trasladar la acumulación a las colas.
- Cada publicación de datos requiere un permiso asociado a su cola de destino para control de capacidad. El consumidor lo devuelve al resolver la entrega. La falta de permisos detiene nuevas publicaciones mediante una espera.
- No se garantiza recuperación de una ejecución ante caídas de procesos o del broker.

Scenarios

Los siguientes escenarios describen los comportamientos principales que debe contemplar la arquitectura.

1. **Carga y procesamiento concurrente.** Un cliente establece una conexión con el gateway y envía registros por lotes. Espera el ACK de cada uno antes de enviar el siguiente. El sistema alimenta las cinco consultas mientras continúa recibiendo datos. El cliente puede recibir y guardar resultados durante el envío.
2. **Clientes con entradas diferentes.** Dos o más clientes envían un mismo conjunto de artículos al mismo tiempo. Cada ejecución conserva sus propios acumuladores y resultados. Una coincidencia de palabras, un conteo o un promedio de un cliente no modifica las respuestas de otro, aunque compartan procesos y colas.
3. **Resultados individuales de Q1 y Q2.** Un artículo histórico en inglés permite emitir su URL y fecha de modificación. Si contiene una sección biográfica, también puede producir un resultado de Q2 después de descargar la imagen. La respuesta de Q1 no depende de que termine esa descarga, puede devolver resultados antes. Q2 transmite imágenes fragmentadas para respetar el tamaño del mensaje.
4. **Resultados de Q3.** Una palabra aparece primero en un abstract en inglés y posteriormente en uno en francés. Intersección registra ambas presencias y agrega esa palabra a un lote de resultados, el cual se publica en cuanto se complete según su tamaño máximo configurado o cuando finaliza la entrada, pero no necesariamente de inmediato. Sus apariciones posteriores no generan nuevas emisiones para esa ejecución. Si solo aparece en un idioma, no se devuelve.
5. **Obtención del top global en Q4.** Las instancias de Frecuencia Palabras envían conteos parciales por lotes. Top N suma los correspondientes a una misma palabra y espera a completar todos los aportes y cierres antes de emitir el top definitivo. El desempate es por palabra ascendente.
6. **Candidato de Q5 recibido antes del promedio.** Un artículo reciente llega a Comparación cuando todavía no se conoce el promedio histórico definitivo. Su URL y conteo se conservan en un buffer, el cual es volcado a disco en un archivo de pendientes al alcanzar un cierto umbral definido. Cuando se dispone del promedio y se acabaron de recibir entradas nuevas, se evalúan los pendientes y se publican los que lo superan.
7. **Etapa de procesamiento más lenta que la entrada.** Si una etapa no puede admitir nuevos datos por saturación, la falta de permisos bloquea publicaciones nuevas. El gateway puede demorar la confirmación de nuevos lotes, mientras el cliente continúa

recibiendo resultados. No se descartan registros para aliviar la carga. En este caso todas las ramas se frenan mutuamente dado que se dejaron de recibir nuevas entradas.

8. **Finalización o terminación local.** En una ejecución exitosa, el gateway entrega los resultados pendientes y comunica el término de las cinco consultas. Ante un error local o SIGTERM, el proceso afectado libera sus recursos y termina, sin garantizar resultados completos.
9. **Validación contra la referencia serial.** Después de completar las ejecuciones, se ejecuta el notebook de referencia sobre los mismos archivos de entrada y se comparan las salidas por cliente y consulta, incluyendo los bytes de las imágenes descargadas.

Size and Performance

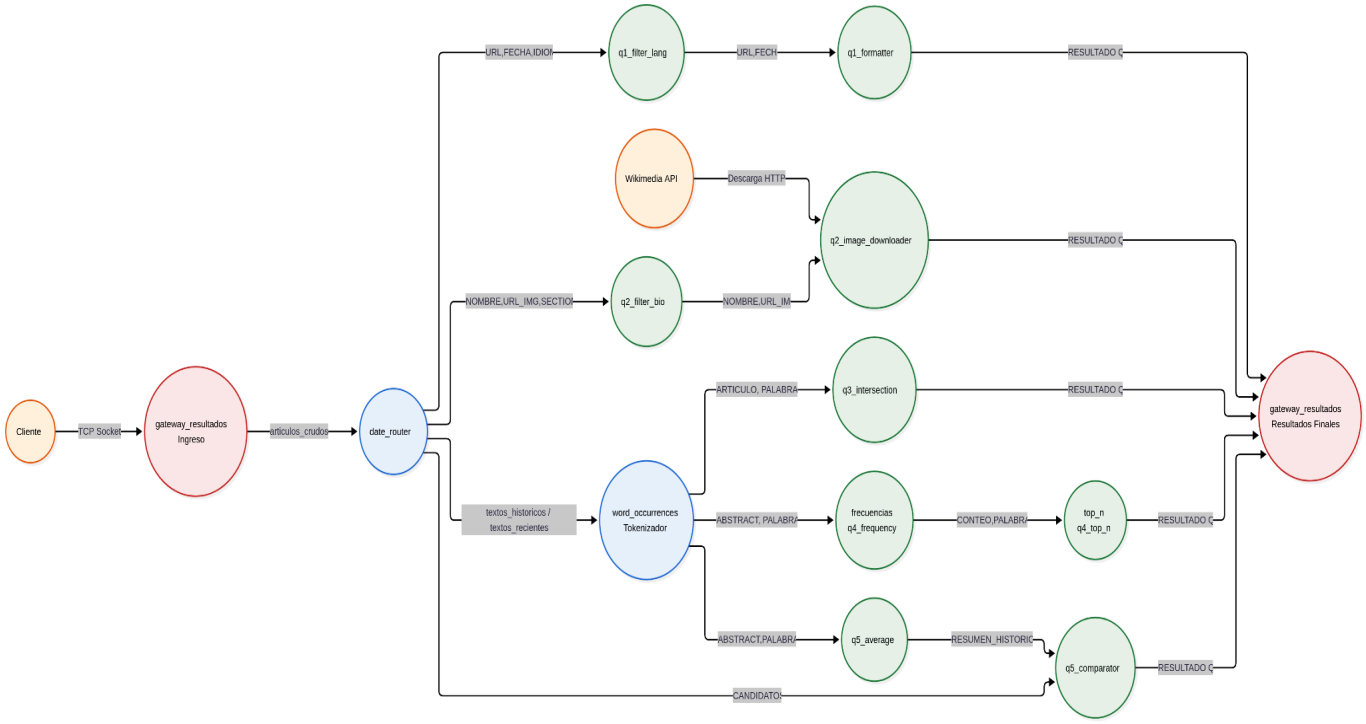
El costo del procesamiento depende de los artículos admitidos, el tamaño de sus abstracts, el vocabulario distinto, la cantidad de clientes y las imágenes descargadas. La lectura del cliente es incremental. La etapa compartida de palabras tokeniza cada abstract una vez: produce palabras únicas para Q3 y Q4 y conserva la cantidad total de palabras no triviales, incluyendo repeticiones, para Q5.

Q3 conserva en memoria la presencia por idioma de las palabras asignadas a su instancia. Frecuencia mantiene parciales por ejecución y los emite según el presupuesto del mensaje. Top N utiliza un buffer de consolidación y archivos ordenados que fusiona incrementalmente; la selección final conserva hasta veinte candidatos. Average mantiene suma y cantidad por ejecución. Comparator acumula candidatos en buffers y archivos separados por ejecución, que recorre después del cierre de sus dos entradas.

La capacidad de las colas de datos se regula mediante permisos y confirmaciones. Estos límites no representan un límite global de memoria del sistema: el costo agregado depende de la cantidad de colas, réplicas, clientes y estados activos.

Los posibles cuellos de botella incluyen la tokenización, el gateway, el broker, la consolidación de una ejecución en Top N y las descargas externas de Q2. Aumentar réplicas puede mejorar algunas etapas, pero no elimina restricciones externas ni garantiza un incremento proporcional del rendimiento.

DAG



El DAG representa el recorrido de los datos desde su ingreso hasta la obtención de los resultados de las cinco consultas. El cliente envía lotes mediante TCP al gateway, que identifica la ejecución y publica los artículos en el sistema. El gateway aparece separado en ingreso y recepción de resultados para distinguir ambas responsabilidades, aunque pertenecen al mismo servicio. Las conexiones entre las etapas internas representan comunicaciones mediante el middleware y RabbitMQ.

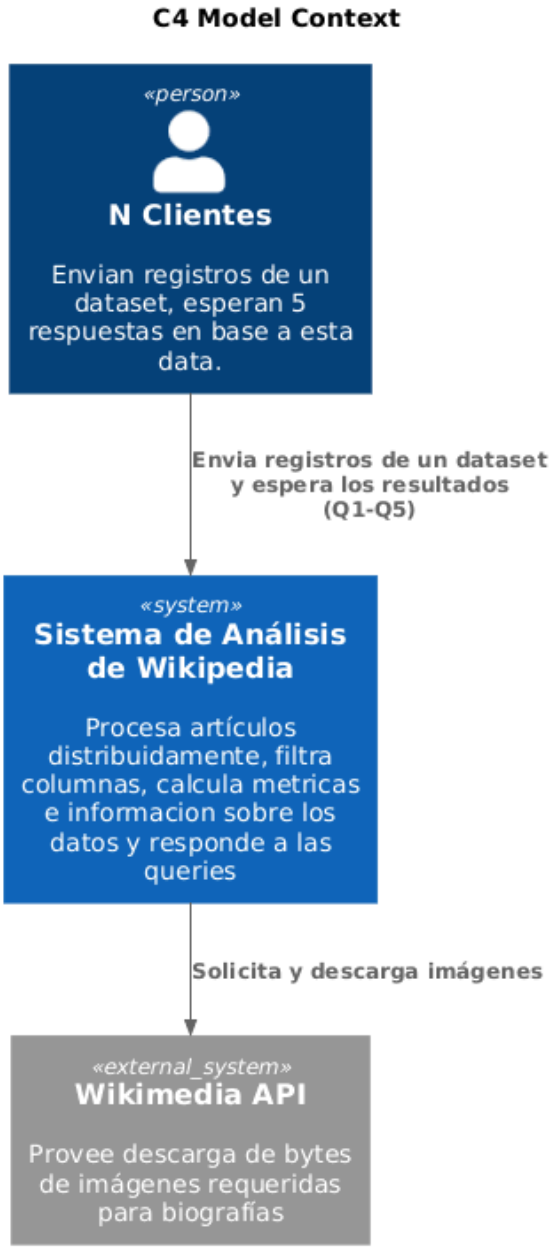
El nodo **date_router** clasifica los artículos por fecha y conserva los campos necesarios para cada destino. Los artículos hasta 2020 inclusive alimentan las ramas Q1 y Q2: **q1_filter_lang** selecciona los escritos en inglés y **q1_formatter** prepara su salida; **q2_filter_bio** selecciona los artículos con secciones biográficas y **q2_image_downloader** obtiene las imágenes de Wikimedia y publica sus fragmentos.

Los textos históricos y recientes llegan a **word_occurrences**, que comparte la tokenización y la eliminación de stopwords y números entre Q3, Q4 y Q5. Para Q3 produce pares de palabra e idioma; para Q4, palabras sin repeticiones dentro de cada abstract; y para Q5, cantidades de palabras no triviales que sí incluyen repeticiones. Los artículos recientes generan candidatos con su URL y cantidad de palabras.

En Q3, **q3_intersection** identifica las palabras presentes en ambos idiomas y reúne las coincidencias en lotes de resultados, evitando emitirlas nuevamente dentro de una ejecución. En Q4, las instancias de Frecuencia producen conteos parciales que **top_n** consolida antes de seleccionar las veinte palabras más frecuentes. En Q5, **q5_average** obtiene la suma y cantidad de artículos históricos, mientras **q5_comparator** almacena los candidatos recientes y espera el cierre de sus dos entradas para publicar aquellos que superan el promedio.

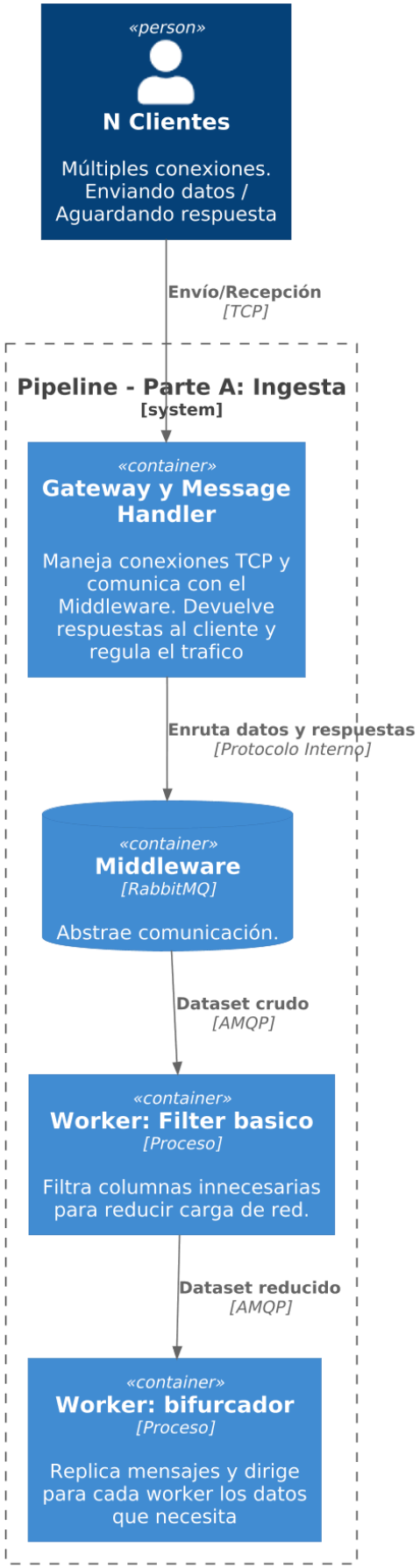
Las etapas avanzan concurrentemente, pero algunas necesitan completar sus entradas antes de emitir sus resultados: Top N requiere los totales globales, Average necesita todos los aportes históricos y Comparator espera tanto los candidatos como el resumen histórico. El middleware coordina estos cierres mediante EOF y contadores de lotes resueltos. Finalmente, el gateway devuelve los resultados al cliente correspondiente y comunica la finalización de la ejecución.

Diagrama de Contexto y Contenedores (C4 Model)

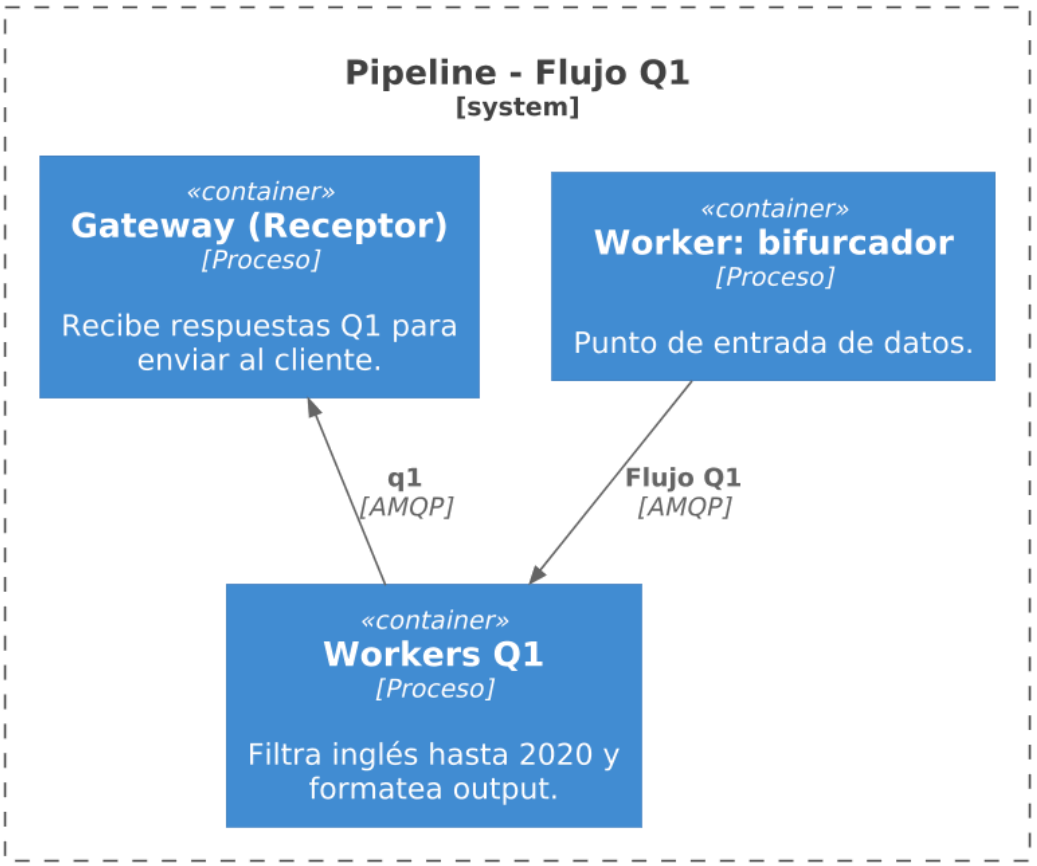


Existiran 0..N clientes, procesos independientes a nuestro sistema clientes.El sistema comenzará a recibir y procesar los registros que el cliente envíe, se aplicaran operaciones de filtros temprano para reducir la sobrecarga de red y facilitar el tiempo de resolución de las consultas.La idea es permitir tener **parámetros** que permitan para fomentar la escalabilidad del sistema, en diversos cuellos de botella.
Además se nombra un sistema externo, la **API de wikipedia**, la cual es necesaria para resolver una de las consultas pedidas que involucra la descarga de binarios.

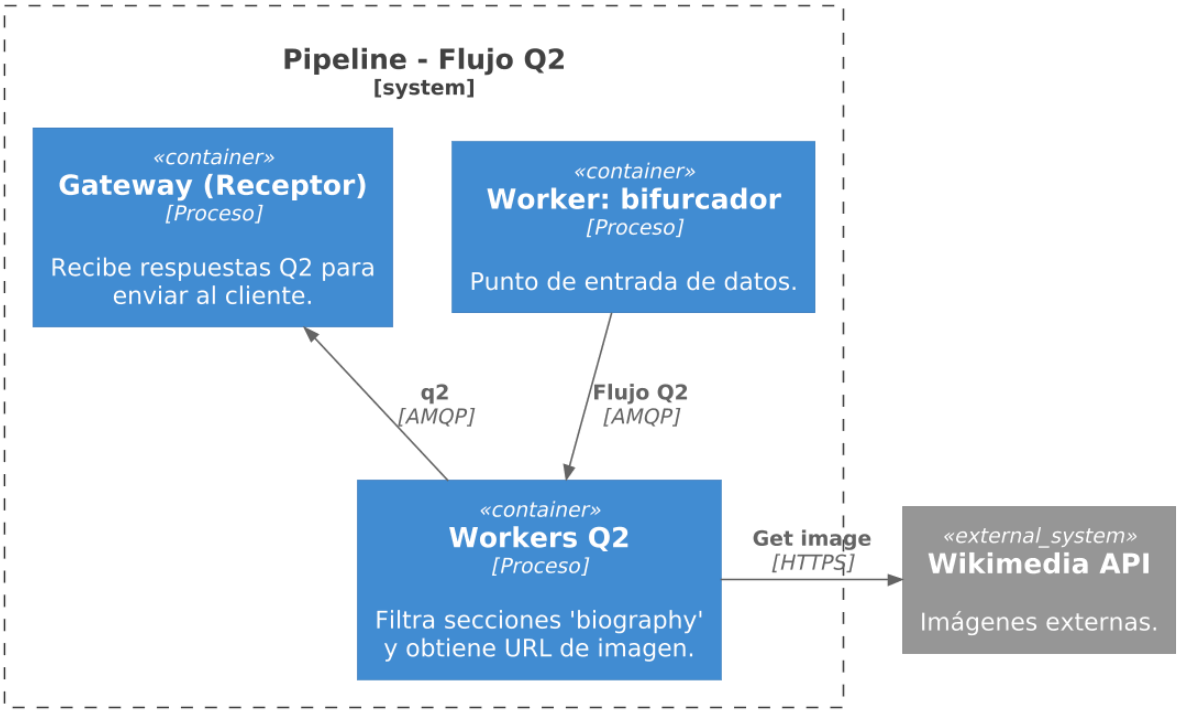
C4 MODEL C2 CONTAINER



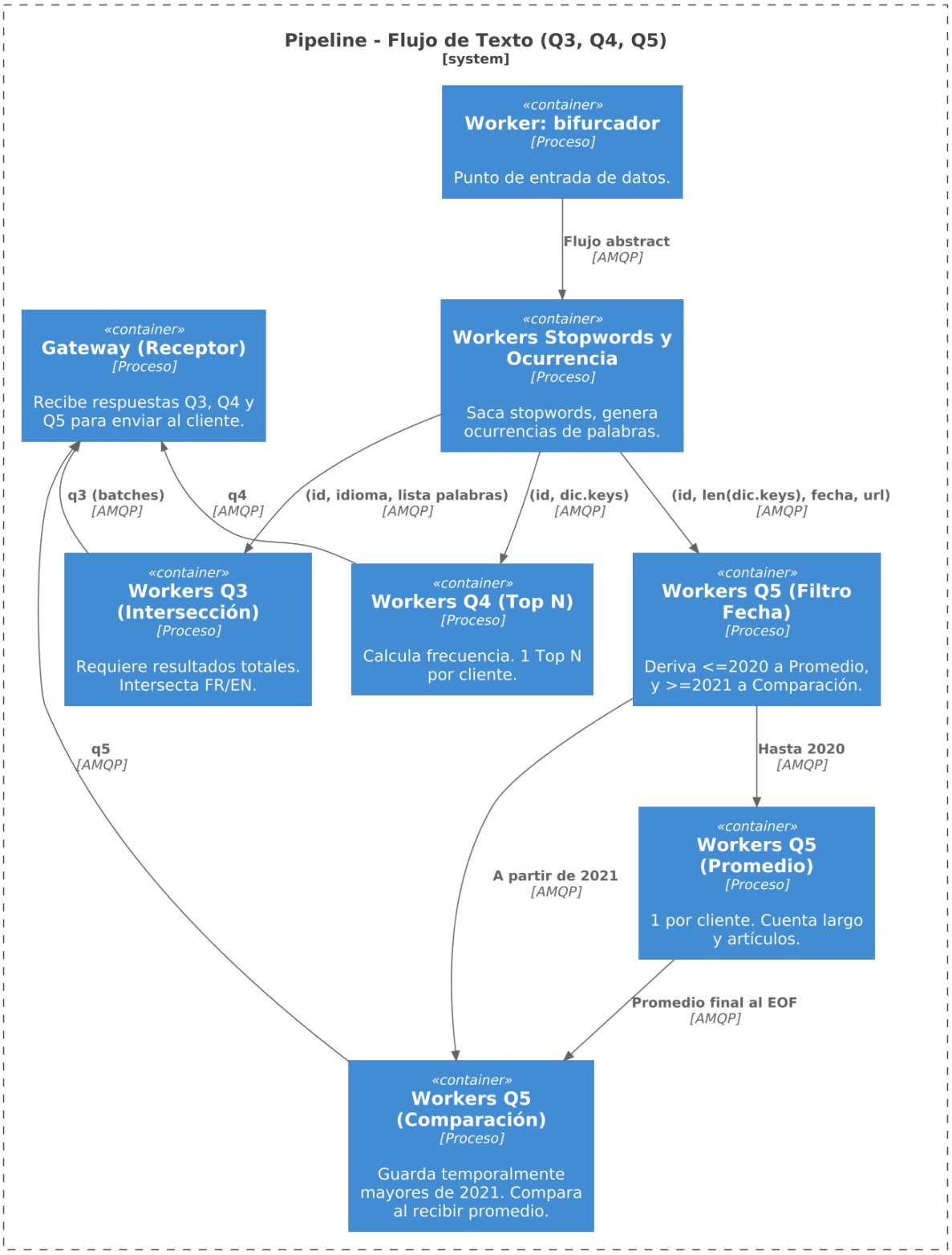
C4 MODEL C2 Q1



C4 MODEL C2 Q2



C4 MODEL C2 Q3,Q4,Q5



En el diagrama de contenedores podemos ver todos los contenedores que se estarán ejecutando en un ciclo de vida común. Nótese a los contenedores según el contexto de c4 y no como contenedores docker o similares, el término de contenedor refiere a procesos que están ejecutándose en una computadora. Se desarrolla la operación de cada contenedor interno al sistema.

En este caso no se visualizan casos relacionados con problemas de concurrencia, estos quedan delegados para otros tipos de diagrama que veremos más adelante.
El Gateway atiende los sockets TCP de los clientes y enruta la data hacia el Middleware, que abstrae toda la comunicación. El primer orker hace una limpieza temprana descartando todas las columnas que por conocimiento del problema no van a ser necesarias en ningún momento.
Un nodo actúa como bifurcador y replicador enviando a cada Worker solo la información que necesita para operar, de esta manera evitamos transmitir información innecesaria y sobrecargar la red. A costa de generar más mensajes, pero favorecer la distribución replicamos mensajes, esto es un **trade off.**
El flujo que opera con los abstract (Q3, Q4, Q5) avanza hacia Workers que limpian las stopwords y arman los diccionarios de frecuencias descartando todo lo demás. Q3 hace la intersección de idiomas y devuelve los resultados en batches para no ahogar al cliente, ya que en este caso podemos obtener resultados parciales válidos y no esperar a tener un único resultado, **este batch piensa ser parametrizable.** Q4 tiene que esperar obligatoriamente el EOF (End of File) para recién armar el top 20 final por cliente. Q5 splitea el flujo en dos por año para poder calcular el promedio y hacer la comparación correcta, por lo que también necesitará obtener primero todo el flujo de datos del cliente.

Componente

Se adjunta el diagrama de C3 componente para el caso donde hace sentido explicar en detalle el funcionamiento interno, más allá de lo expresado en C2.

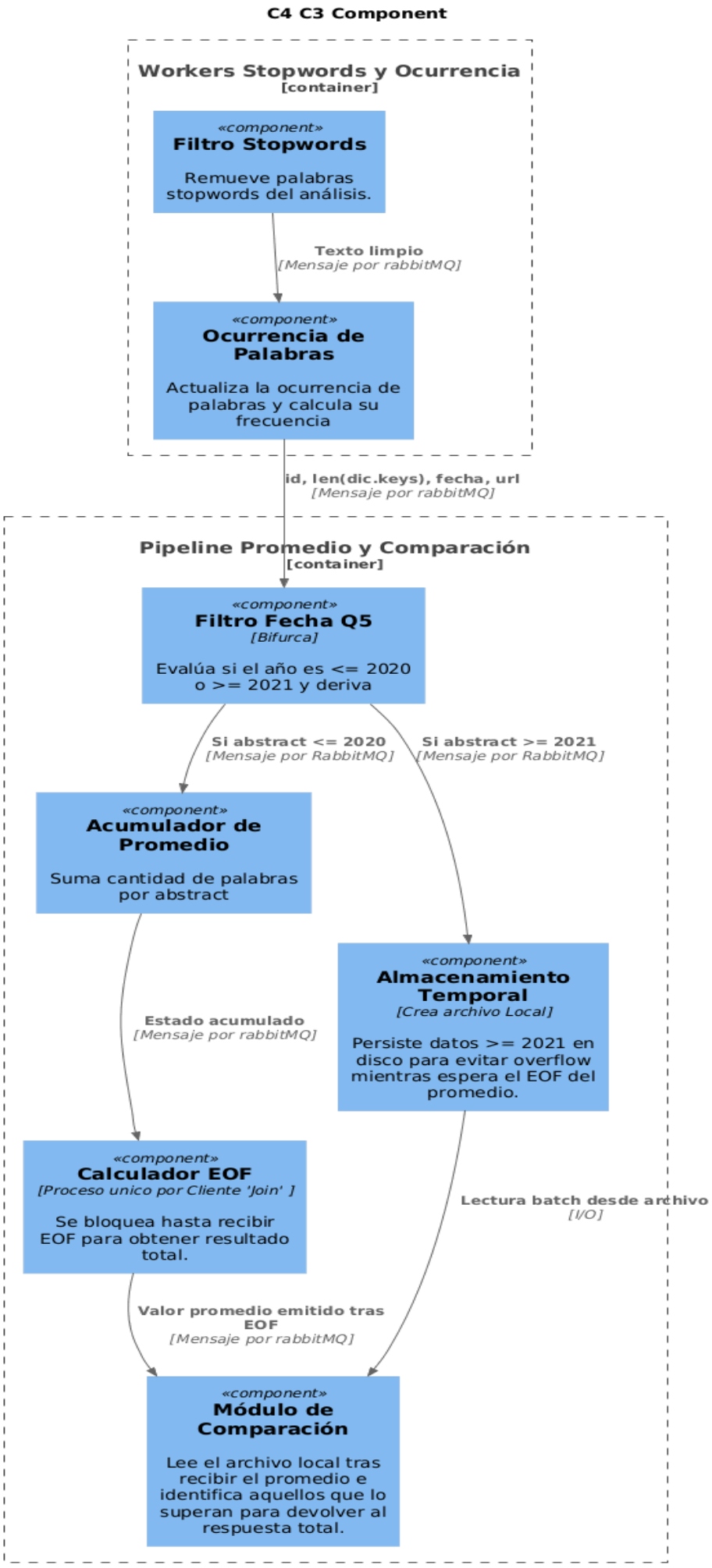
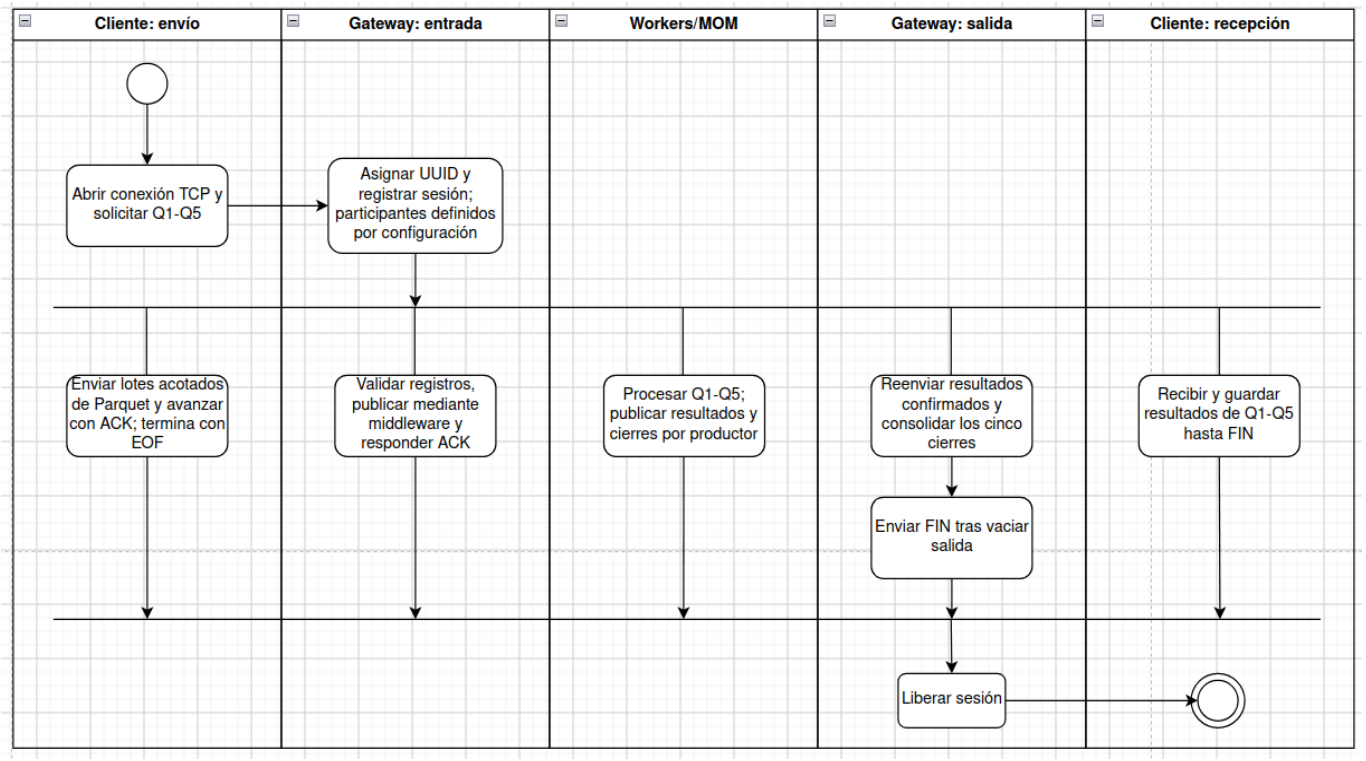

C4 C3 Component
Workers Stopwords y Ocurrencia
[container]
«component»
Filtro Stopwords
Remueve palabras stopwords del análisis.
Texto limpio
[Mensaje por rabbitMQ]
«component»
Ocurrencia de Palabras
Actualiza la ocurrencia de palabras y calcula su frecuencia
id, len(dic.keys), fecha, url
[Mensaje por rabbitMQ]
Pipeline Promedio y Comparación
[container]
«component»
Filtro Fecha Q5
[Bifurca]
Evalúa si el año es <= 2020 o >= 2021 y deriva
Si abstract <= 2020
[Mensaje por RabbitMQ]
Si abstract >= 2021
[Mensaje por RabbitMQ]
«component»
Acumulador de Promedio
Suma cantidad de palabras por abstract
«component»
Almacenamiento Temporal
[Crea archivo Local]
Persiste datos >= 2021 en disco para evitar overflow mientras espera el EOF del promedio.
Estado acumulado
[Mensaje por rabbitMQ]
«component»
Calculador EOF
[Proceso unico por Cliente 'Join']
Se bloquea hasta recibir EOF para obtener resultado total.
Lectura batch desde archivo
[I/O]
Valor promedio emitido tras EOF
[Mensaje por rabbitMQ]
«component»
Módulo de Comparación
Lee el archivo local tras recibir el promedio e identifica aquellos que lo superan para devolver al respuesta total.

Diagrama de Actividades

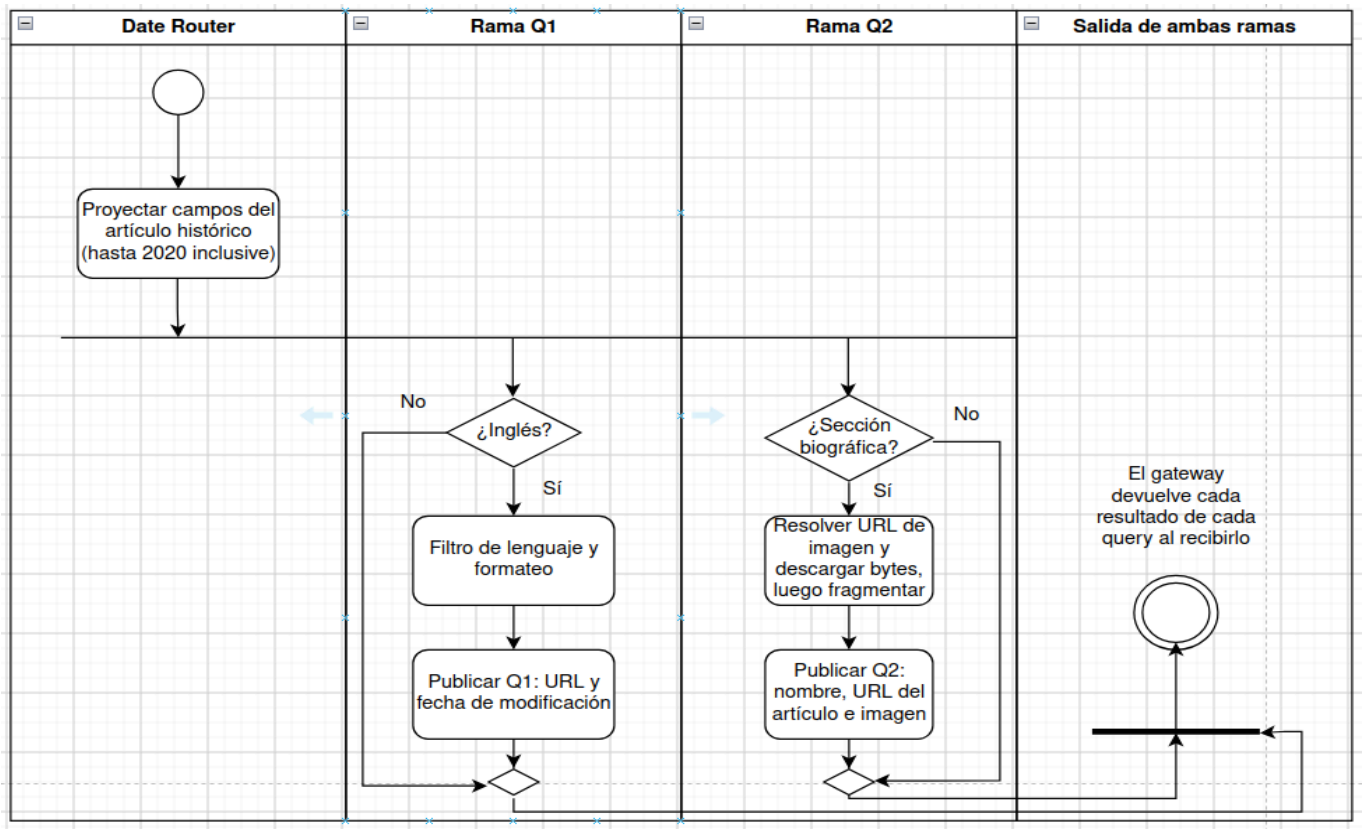
A1: una ejecución, envío y resultados concurrentes



El diagrama muestra una ejecución desde la apertura de la conexión hasta la liberación de la sesión. El cliente solicita las cinco consultas y el gateway le asigna un UUID único y registra su sesión, aunque los participantes ya estén definidos previamente en la configuración del sistema. La primera barra habilita cinco actividades concurrentes: enviar datos, admitirlos y distribuirlos, procesar las consultas, reenviar resultados y recibirlos.

El cliente lee los archivos parquet y envía lotes acotados con los artículos, espera una confirmación de admisión —ACK— para avanzar. Al terminar la carga envía EOF para avisar que ya terminó. Los Workers procesan las consultas y publican resultados y cierres por productor. El gateway reenvía los resultados mientras consolida la finalización de las cinco consultas; cuando no quedan resultados pendientes de envío, emite FIN. El cliente recibe y guarda resultados hasta recibir ese mensaje. La barra inferior reúne la finalización de las actividades y permite liberar la sesión. Los carriles de entrada y salida distinguen responsabilidades del mismo gateway, al igual que los de envío y recepción del cliente.

A2: Query 1 y Query 2, procesamiento de un histórico



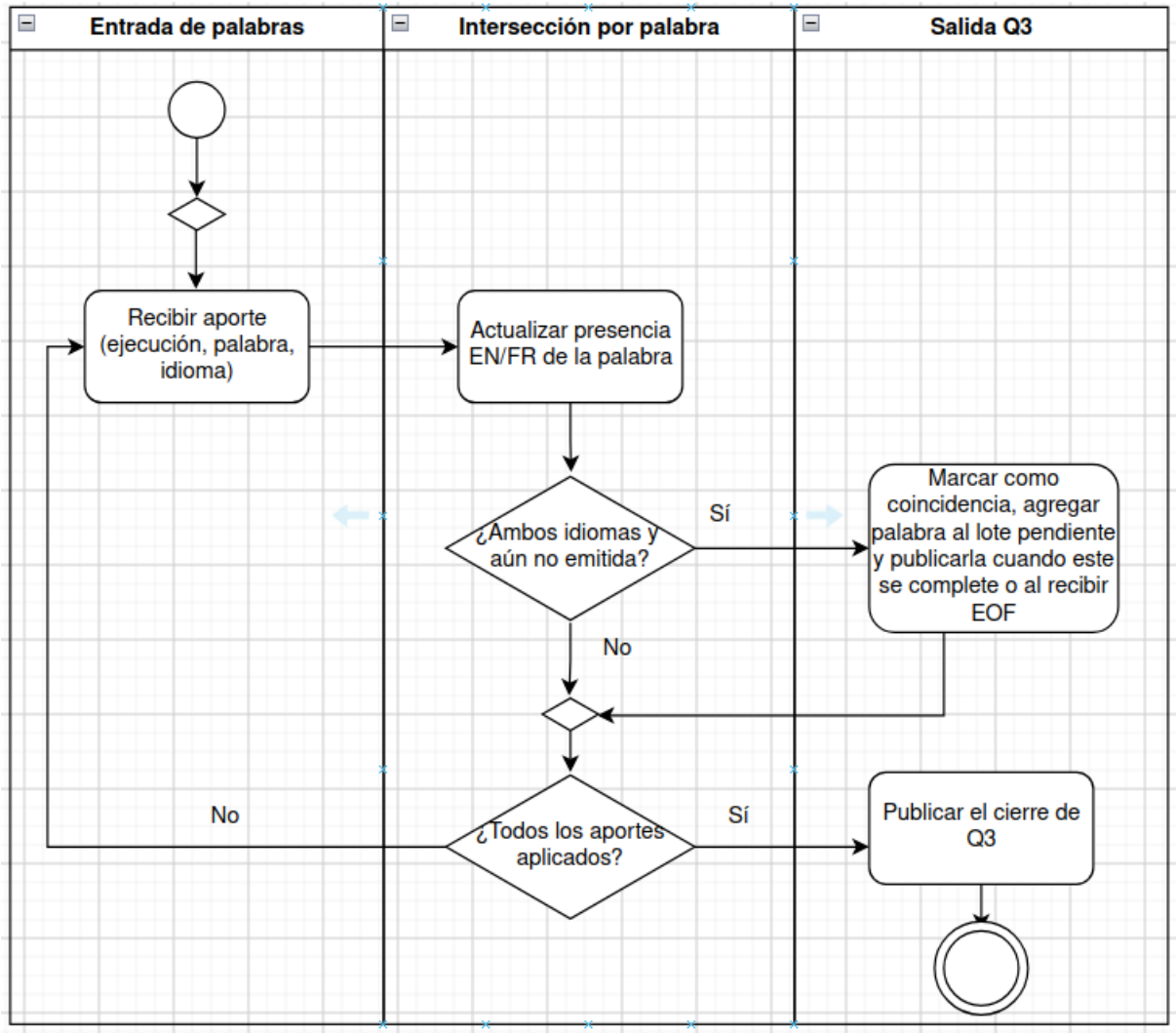
El diagrama describe el procesamiento de un artículo histórico, correspondiente a una fecha hasta 2020 inclusive. Después de que el nodo de date_router proyecte sus campos, la barra de bifurcación habilita dos evaluaciones concurrentes. La rama Q1 verifica si el artículo está escrito en inglés con un filtro por lenguaje y, cuando corresponde, publica su URL y fecha de modificación luego de un formateo. La rama Q2 verifica si contiene una sección biográfica; si la contiene, obtiene la URL de la imagen, descarga sus bytes y publica el nombre, la URL del artículo y la imagen.

Cuando una condición no se cumple, la rama termina sin publicar un resultado. Los rombos inferiores reúnen los caminos alternativos de cada rama y la barra final espera que ambas concluyan. Esa sincronización completa el procesamiento del artículo, pero no retrasa las publicaciones realizadas previamente: Q1 puede entregar su resultado mientras Q2 continúa descargando la imagen.

Alguno de los siguientes escenarios pueden ocurrir durante la descarga de la imagen:

- Descarga exitosa (caso común): publicar los fragmentos.
- No hay URL de imagen: publicar artículo sin imagen.
- Error 404/410 en la descarga: se omite ese artículo.
- Error transitorio (como un 429): esperar y reintentar.
- Reintentos agotados sin éxito: publicar artículo sin imagen.

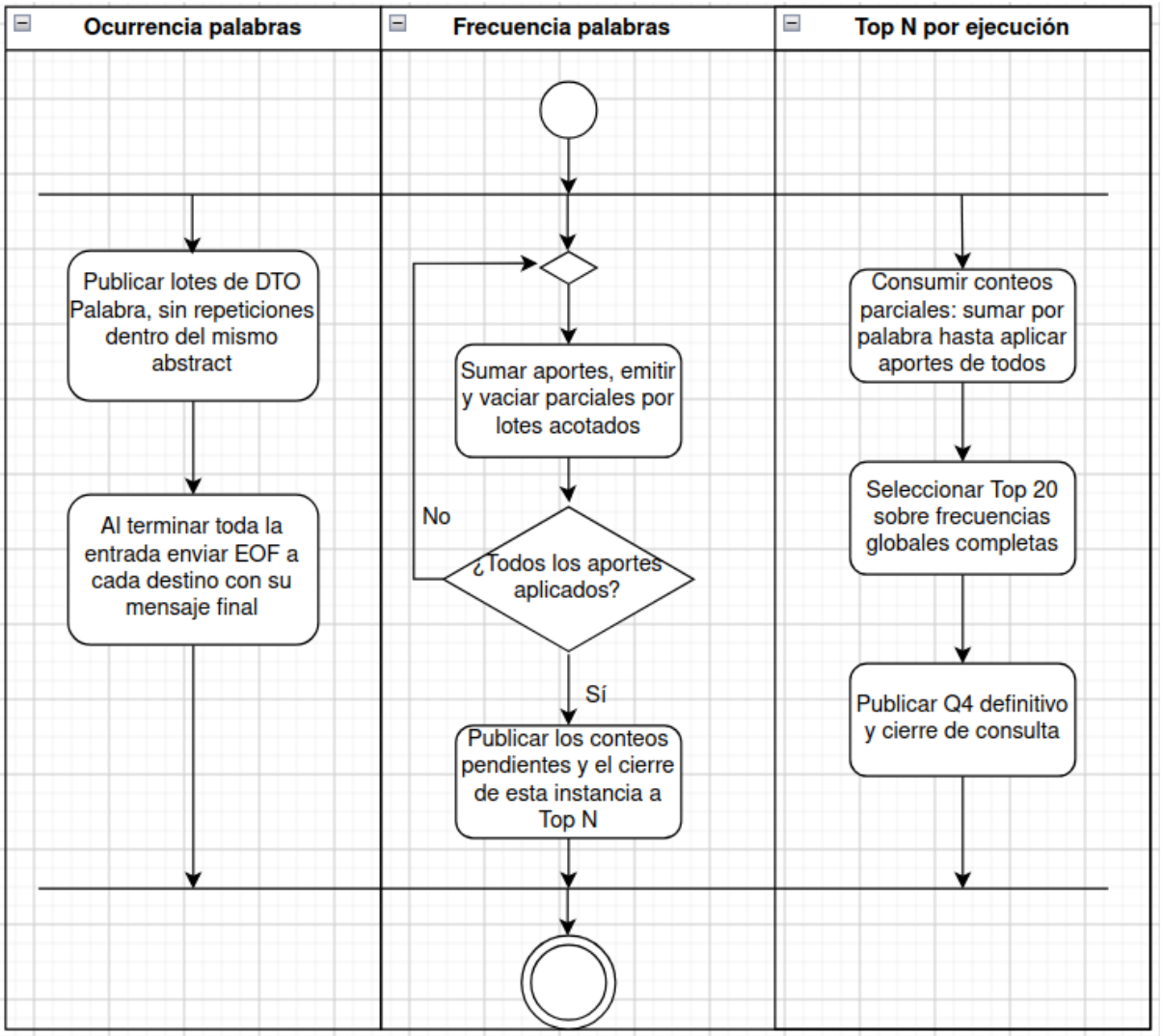
A3: Query 3, palabra compartida y emisión temprana



El diagrama muestra un ciclo de recepción de palabras de los artículos. Cada aporte identifica la ejecución, una palabra y su idioma. Al recibirlo, se actualiza la presencia de esa palabra en inglés o francés. Si ya está presente en ambos idiomas y todavía no fue emitida, se marca como coincidencia y se agrega al lote de palabras que se esté construyendo en ese momento. Cuando este lote esté completo según el tamaño máximo definido o se reciba EOF entonces se publican las coincidencias de ese lote, pero esto no necesariamente ocurre inmediatamente en cuanto se marque a la palabra como una coincidencia. Si alguna de esas condiciones no se cumple, no se genera una salida.

Luego de procesar una palabra se comprueba si ya se finalizó o no según si se haya recibido EOF o no. En caso de que sí entonces termina el procesamiento de esta query, y en caso de que no se continua con la siguiente palabra.

A4: Query 4, frecuencias parciales y top global

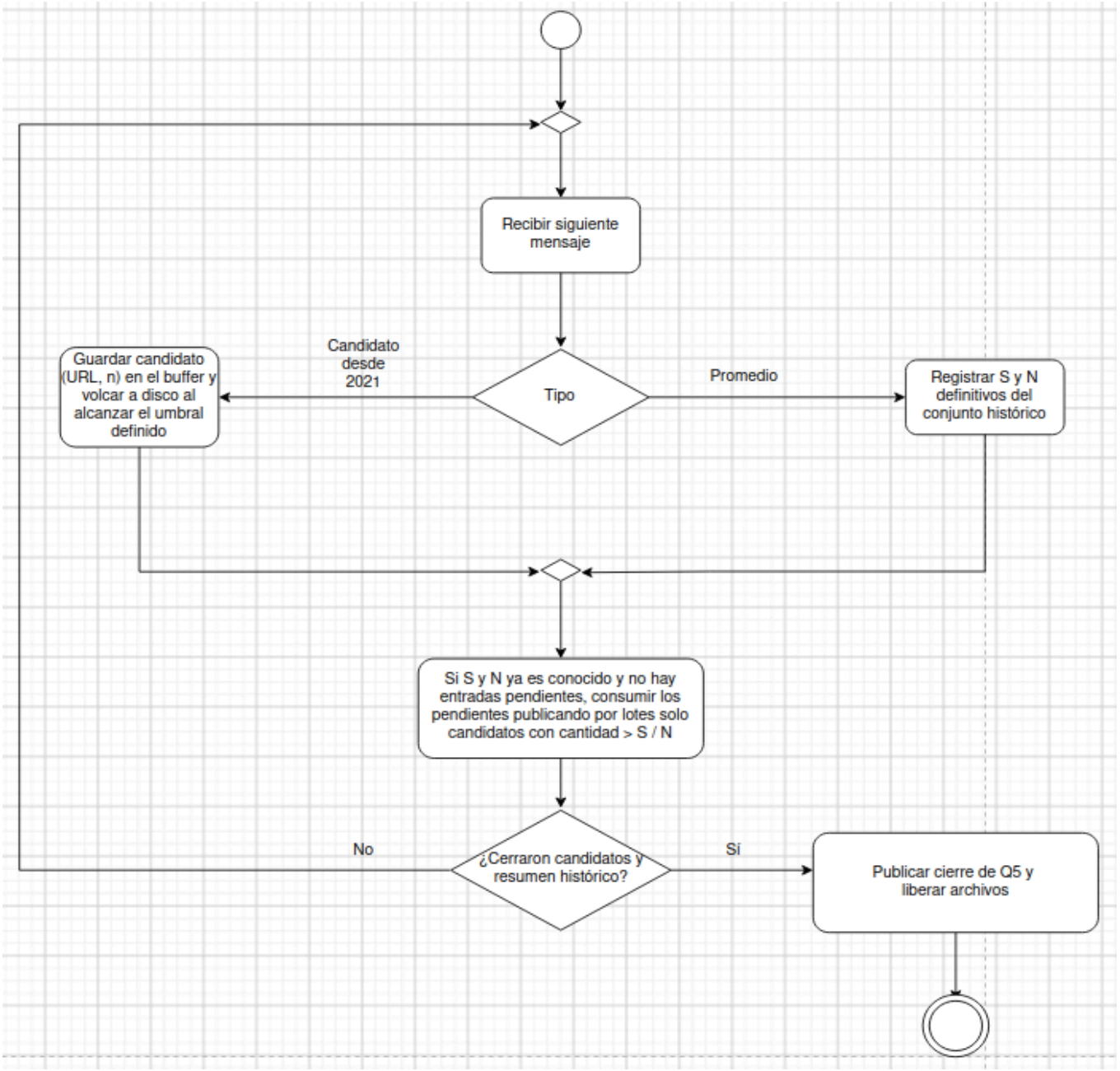


El diagrama representa tres etapas que avanzan concurrentemente. Ocurrencia palabras publica lotes de DTO Palabra sin repeticiones dentro del mismo abstract. Al terminar su entrada, envía un mensaje de fin.

Frecuencia palabras suma las contribuciones recibidas por ejecución y publica conteos parciales por lotes acotados. Al emitirlos, vacía esos acumulados para que los siguientes parciales no vuelvan a incluir las mismas contribuciones. Cuando se aplicaron todos los aportes y cierres esperados, publica los conteos pendientes.

Top N consume los parciales de una ejecución y suma los correspondientes a una misma palabra. Solo después de completar los aportes y cierres de todas las instancias selecciona las veinte palabras con mayor frecuencia y publica el resultado definitivo de Q4. La barra inferior reúne la finalización de las tres etapas. El orden de selección en esta etapa es frecuencia descendente y luego palabra ascendente para desempatar.

A5: Query 5, candidatos y promedio definitivo

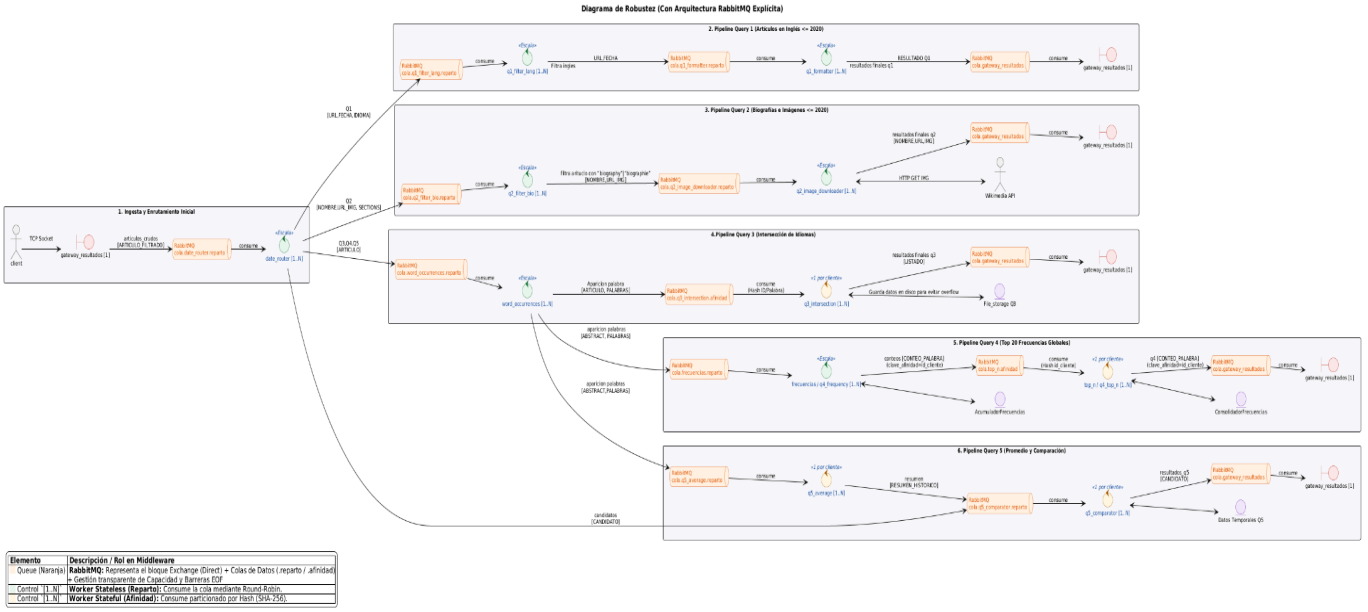


El diagrama describe el procesamiento de mensajes del comparador Q5, con estado separado por ejecución. Se distinguen dos tipos de mensaje: un candidato desde 2021 o la información definitiva del promedio histórico. Los candidatos se guardan como pares **(URL, n)** en un buffer hasta alcanzar un cierto umbral definido y luego se vuelcan a disco en archivos de pendientes, donde n es la cantidad de palabras del abstract. La información histórica se registra mediante S, la suma de cantidades, y N, la cantidad de artículos considerados.

Después de cada mensaje, si se conocen **S** y **N** y ya se dejaron de recibir nuevas entradas, se procesan los candidatos pendientes. Se publican aquellos que cumplen **cantidad > S / N**,

equivalente a superar el promedio. Cuando **N = 0** se usa promedio cero. El ciclo continúa hasta conocer los valores históricos y agotar los pendientes. Entonces se publica el cierre de Q5 y se liberan los archivos.

Diagrama de Robustez



Elemento	Descripción / Rol en Middleware
Queue (Naranja)	**RabbitMQ:** Representa el bloque Exchange (Direct) + Colas de Datos (reparto / afinidad) + Gestión transparente de Capacidad y Barreras EOF
Control [1..N]	**Worker Stateless (Reparto):** Consume la cola mediante Round-Robin.
Control [1..N]	**Worker Stateful (Afinidad):** Consume particionado por Hash (SHA-256).

El diagrama muestra las responsabilidades de los componentes, los recursos de comunicación y los estados utilizados durante el procesamiento. El cliente y Wikimedia son actores externos; el gateway representa la frontera de comunicación con los clientes; los workers implementan los controles de procesamiento; y los acumuladores y archivos representan el estado administrado por esas etapas. Los bloques de RabbitMQ representan recursos alojados en un mismo broker, no servidores independientes.

Durante el ingreso, el gateway recibe los lotes mediante el protocolo TCP, valida los registros y los publica a través del middleware. Las instancias de **date_router** consumen los artículos y proyectan la información necesaria para cada rama. La comunicación interna utiliza colas de reparto cuando los lotes pueden ser procesados por cualquier instancia, y colas de afinidad cuando determinadas contribuciones deben llegar a una misma instancia. El middleware también administra los permisos de capacidad, las confirmaciones y la coordinación de los cierres, cuyos mensajes no se detallan individualmente en el dibujo.

La rama Q1 combina el filtro de idioma con el formateador de resultados. La rama Q2 combina el filtro de secciones biográficas con el descargador de imágenes, que accede a Wikimedia y divide las imágenes en fragmentos para respetar el tamaño máximo de los mensajes. Ambas ramas pueden producir resultados durante la recepción de la entrada.

La etapa compartida **word_occurrences** prepara los aportes para las consultas de palabras. En Q3, las contribuciones de una misma palabra y ejecución se dirigen a una misma instancia de Intersección. Esta conserva en memoria la presencia por idioma y las coincidencias ya

incorporadas a la salida. Los resultados se publican por lotes durante el procesamiento y se vacían los pendientes al completar la entrada.

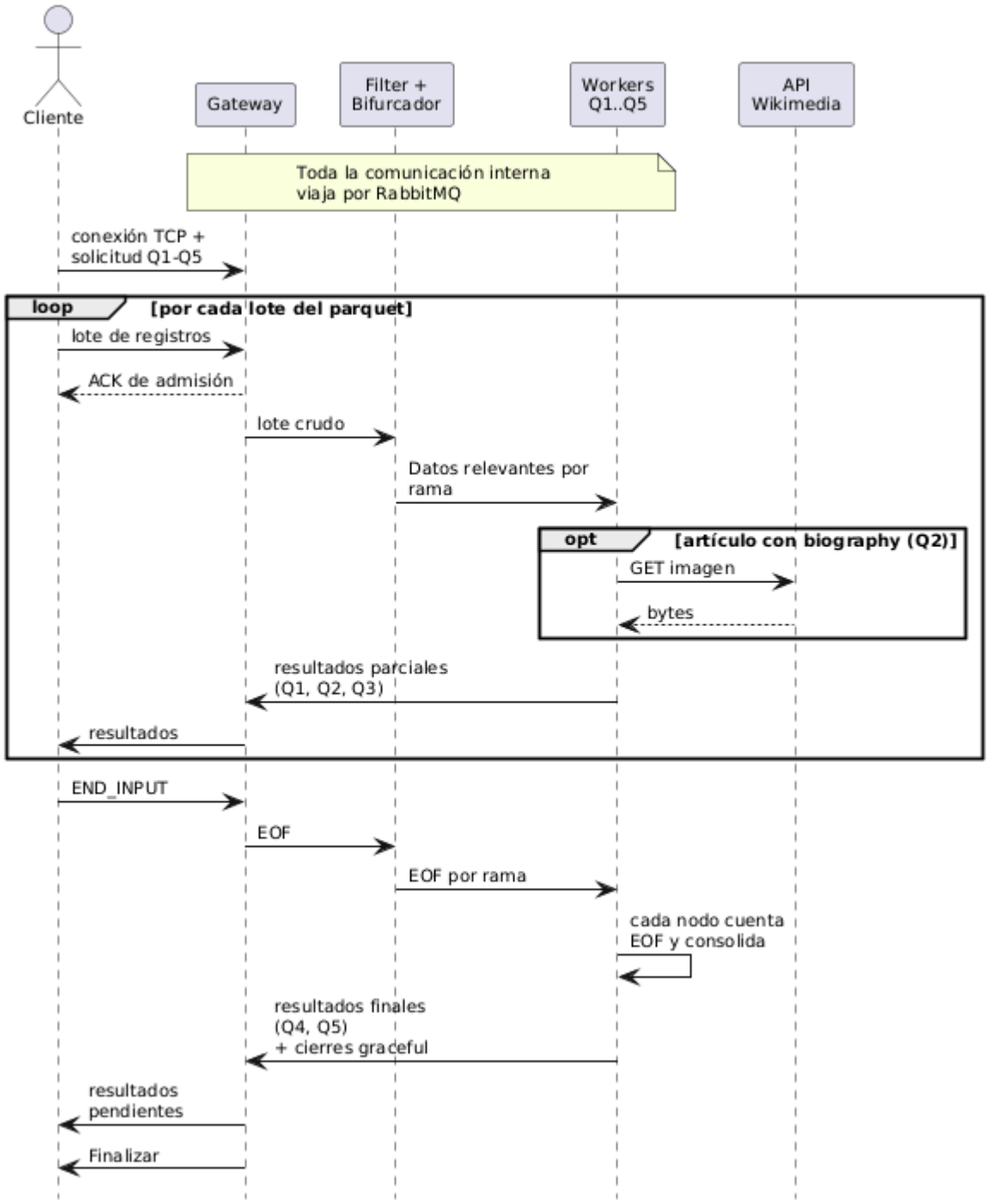
En Q4, Frecuencia mantiene acumuladores por ejecución y publica parciales acotados, retirando las contribuciones ya emitidas. Los parciales de una ejecución se dirigen por afinidad a una instancia de Top N. Su consolidador utiliza un buffer y archivos temporales para obtener los totales sin cargar todo el vocabulario al finalizar. Una vez completada la entrada, selecciona y publica el top definitivo. Los parciales entre Frecuencia y Top N son resultados intermedios internos y no se entregan al cliente como respuesta de Q4.

En Q5, Average conserva la suma de cantidades de palabras y el número de artículos históricos por ejecución. Comparator recibe ese resumen y los candidatos recientes por entradas separadas. Los candidatos se acumulan en un buffer y se vuelcan a archivos temporales; cuando ambas entradas finalizaron, se recorren incrementalmente y se publican los que superan el promedio histórico. Después se liberan los recursos de la ejecución y se propaga su cierre.

Las salidas de las consultas convergen en el rol de recepción de resultados del gateway. Este utiliza el identificador de ejecución para devolver cada resultado a su cliente, manteniendo separados los resultados aunque las ejecuciones compartan workers y recursos de RabbitMQ.

Diagrama de Secuencia

A continuación mostramos como se espera que actúe el sistema. Para mantener los diagramas legibles, la comunicación entre componentes internos no muestra a RabbitMQ como participante, pero se resalta que todos los mensajes viajan a través de sus exchanges y colas, como se explicó anteriormente.

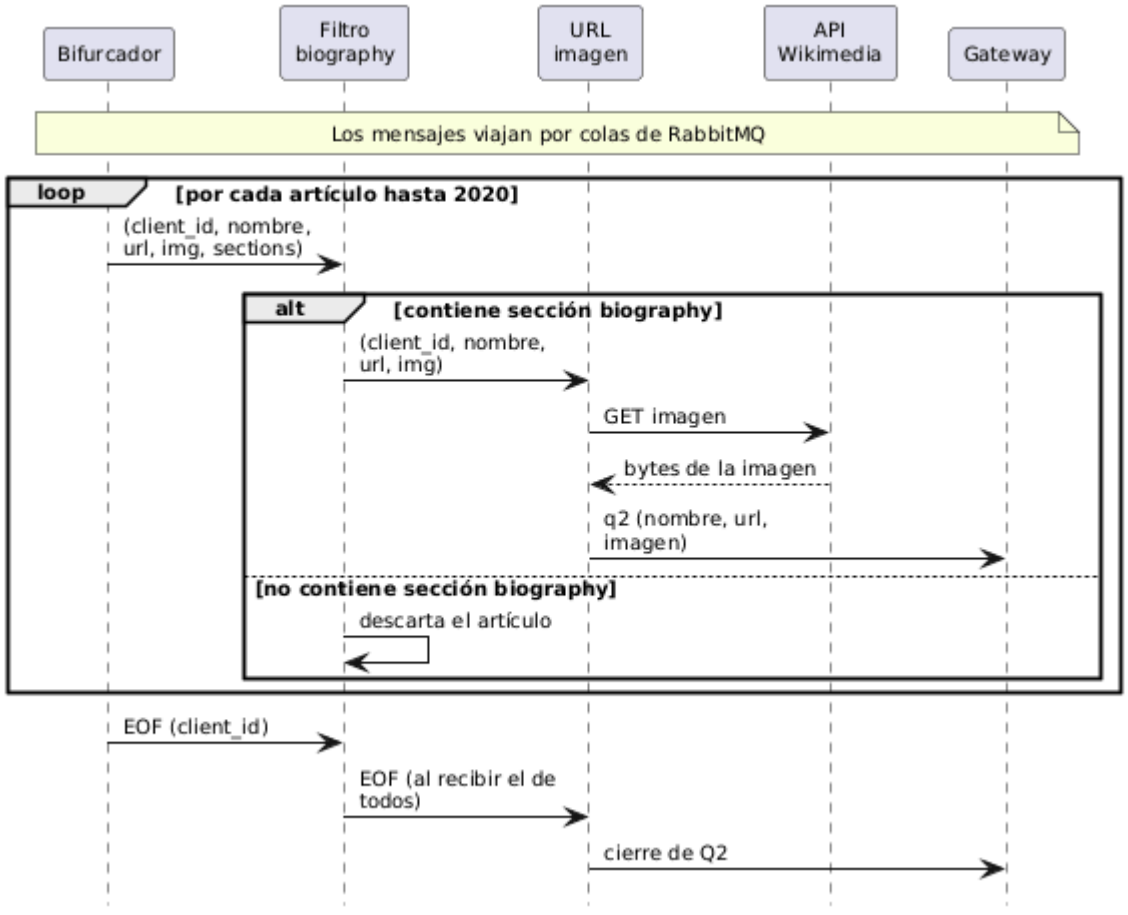


En el diagrama se puede observar el proceso de punta a punta desde que el cliente se conecta y envía los artículos que desea obtener los resultados.

El Gateway registra la petición con un identificador (client_id) y luego se repite un ciclo por cada lote del parquet: el cliente lo envía, el Gateway responde con un ACK, y el lote avanza hacia el Filter y el Bifurcador, que descartan las columnas no usadas y envía los datos necesarios a cada query. Los Workers procesan esos datos y, cuando una query puede emitir resultados parciales (Q1, Q2 y Q3), los envían al Gateway, que los reenvía al cliente mientras este sigue enviando datos. Si un artículo requiere una imagen (Q2), el Worker la descarga de la API de Wikimedia.

Al terminar la carga, el Gateway avisa del EOF que recorre el pipeline. Cada nodo cuenta los EOF recibidos y, al igualar la cantidad de réplicas esperadas, consolida su resultado y propaga el EOF. Así, los resultados que dependen de la totalidad de los datos (Q4 y Q5) se emiten recién al cierre. El Gateway entrega los resultados pendientes y, cuando las cinco queries finalizaron, envía finalizado al cliente.

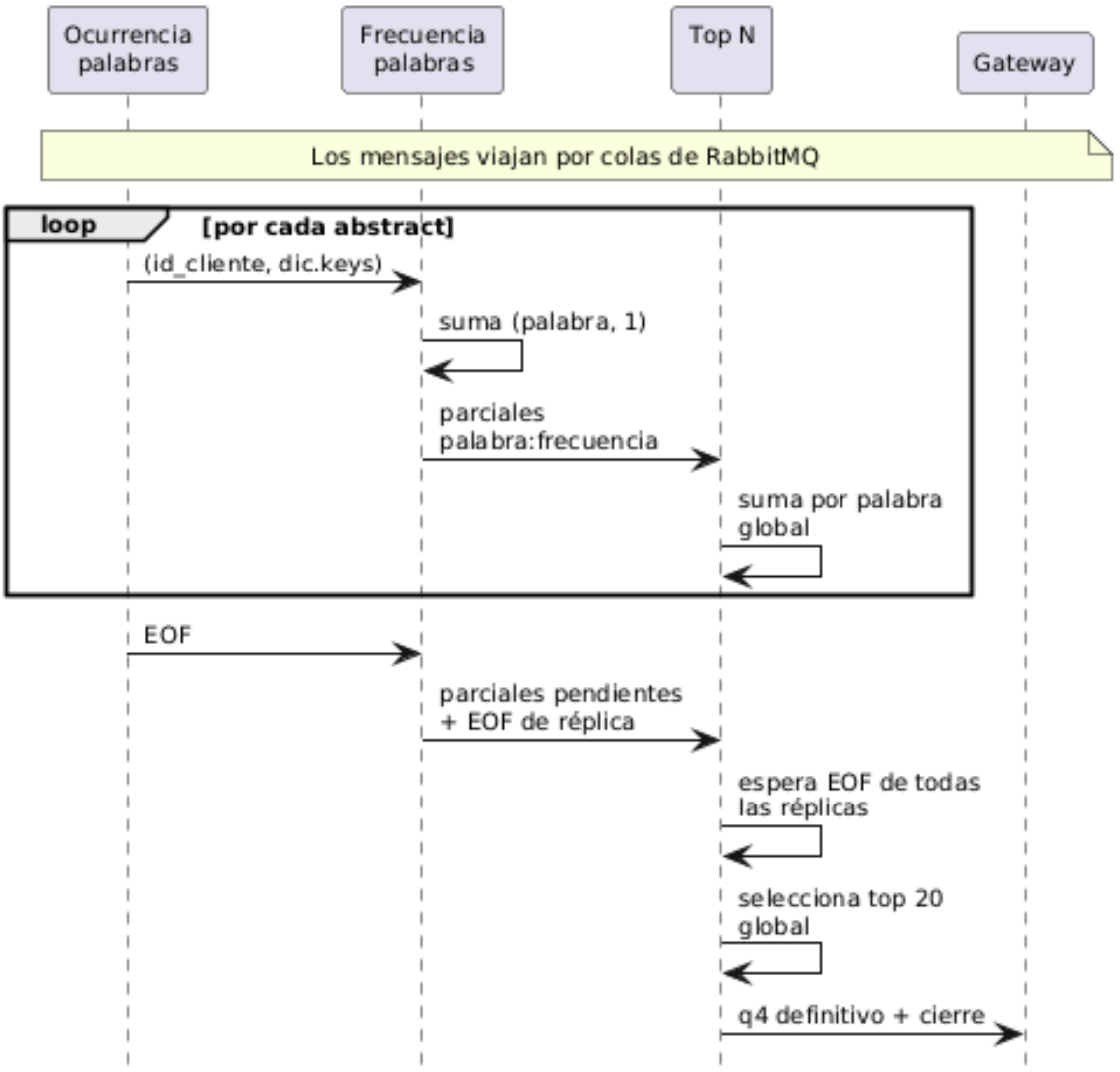
Haciendo detalle en el query Q2:



Este diagrama detalla el procesamiento de Q2 sobre los artículos hasta 2020 inclusive. Por cada artículo, el Bifurcador envía al Filtro biography/biographie los campos nombre, URL, imagen y secciones. Si el artículo contiene una sección, pasa a la etapa de URL imagen, que

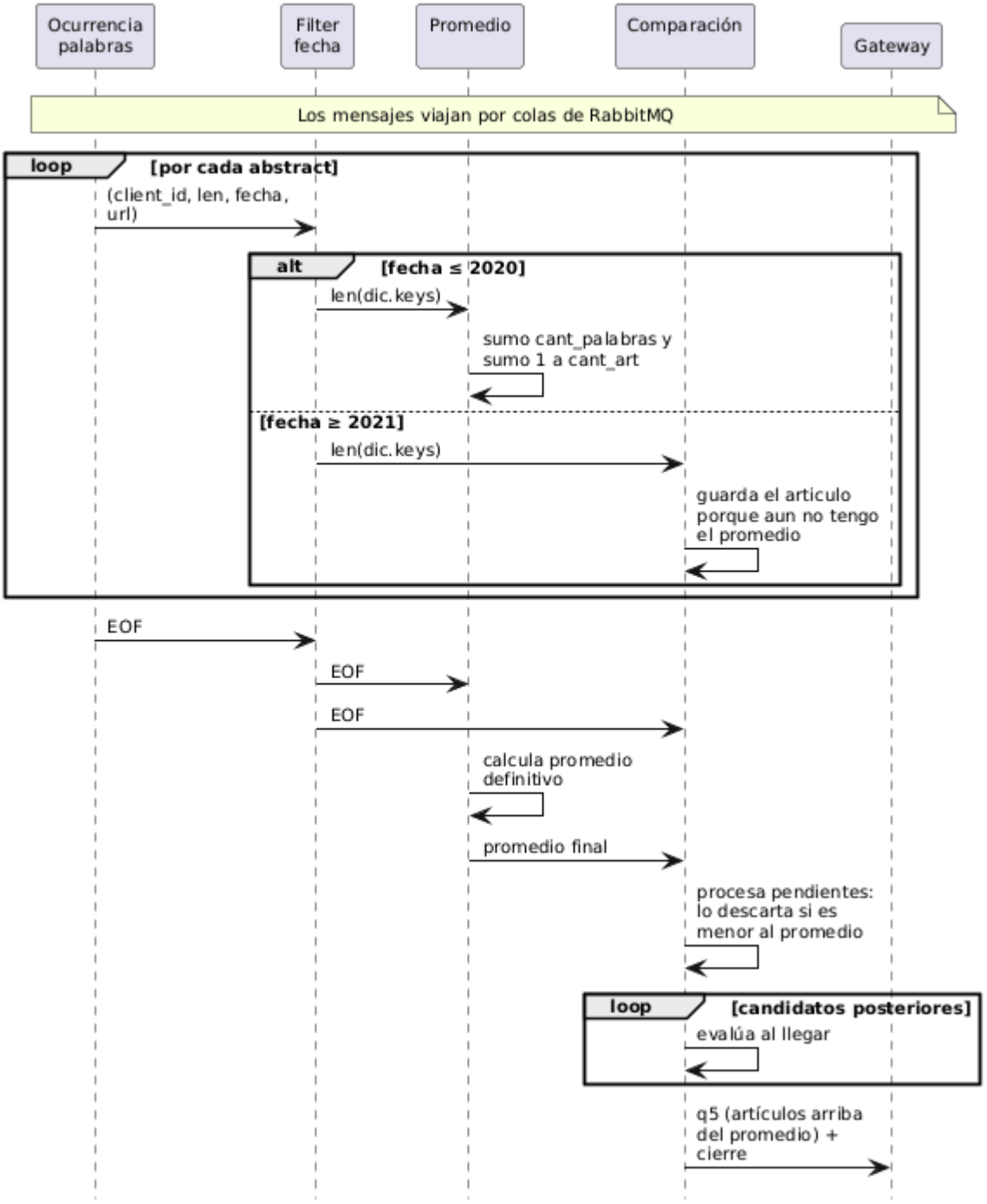
descarga los bytes de la imagen desde la API de Wikimedia y envía al Gateway el nombre, la URL del artículo y la imagen. Si no la contiene, el artículo se descarta sin generar resultado.

Por otro lado el query Q4:



Este diagrama muestra cómo se obtiene el top 20 de palabras más frecuentes. Por cada abstract, Ocurrencia Palabras envía a una instancia de Frecuencia el conjunto de palabras únicas (sin stopwords). Cada instancia suma las apariciones en un acumulador local y, luego se lo comunica a Top N. Top N, suma los parciales correspondientes a una misma palabra, pero no puede emitir hasta conocer todos los artículos. Al terminar la entrada, Ocurrencia envía el EOF y cada instancia de Frecuencia publica sus conteos pendientes junto con su propio cierre, y Top N espera los cierres de todas las réplicas. Recién ahí, selecciona las veinte palabras con mayor frecuencia global y envía al Gateway el resultado definitivo de Q4 junto con el cierre de la query.

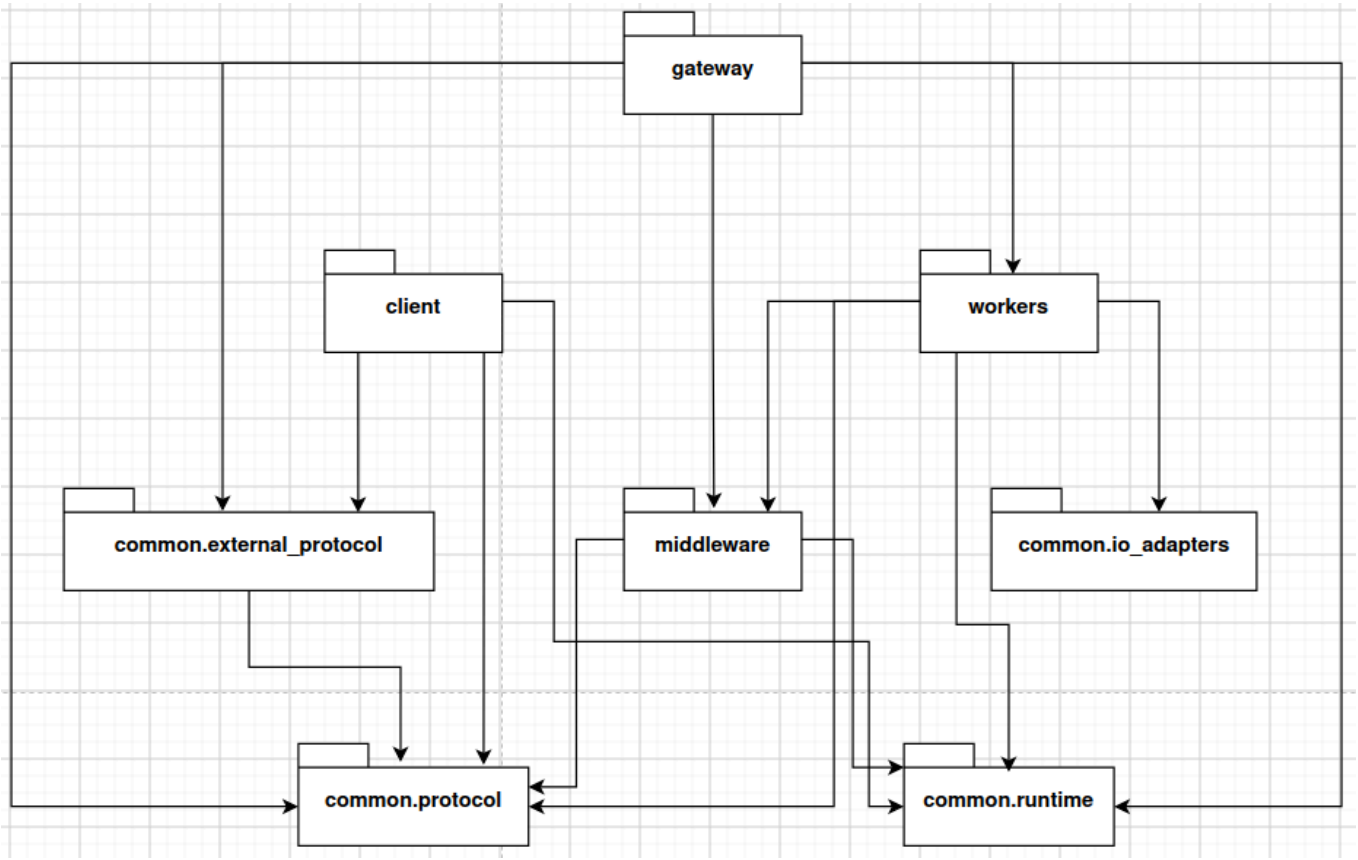
y finalmente el query Q5:



Este diagrama muestra la comparación entre la longitud de los artículos desde 2021 y el promedio histórico de los artículos hasta 2020. Por cada abstract, el Filter fecha recibe su identificador, cantidad de palabras, fecha y URL, y lo rutea según su año: si es hasta 2020, Promedio acumula la suma de cantidad de palabras y la cantidad de artículos. Si es desde 2021, Comparación lo guarda como candidato, ya que todavía no puede saber si supera el promedio. Cuando llega el EOF, Promedio calcula el valor definitivo y envía el promedio definitivo a Comparación, que recién entonces puede evaluar. Procesa primero los pendientes y publica los artículos que cumplen. Los candidatos que lleguen después se evalúan directamente al recibirlos. Una vez recibido el EOF, Comparación envía al Gateway el cierre de Q5.

Diagrama de Paquetes

Paquetes: aplicaciones y código compartido



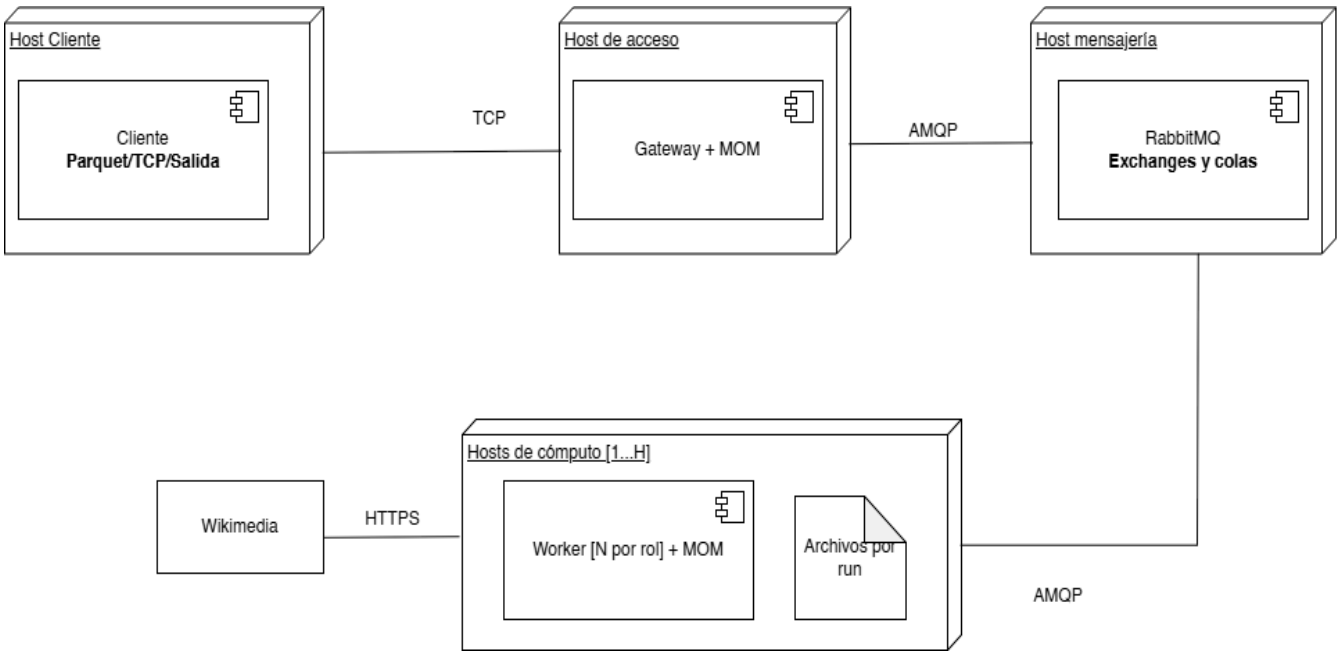
El diagrama presenta la organización general del código y sus dependencias. client contiene la lectura de Parquet, conexión, envío y escritura de resultados; gateway, la gestión de sesiones TCP, validación de artículos, publicación y retorno de resultados.; y workers, los roles de procesamiento (hay un worker común del que parten todos y luego workers especializados en cada rol específico). Estos paquetes utilizan responsabilidades comunes: io_adapters reúne las descargas HTTP y operaciones completas sobre archivos.; middleware, la configuración de comunicación por grupos, RabbitMQ, capacidad, EOF y recursos por ejecución.

external_protocol (es un módulo, no un paquete) contiene los frames TCP entre cliente y gateway, luego protocol los DTO, esquemas, primitivas y serialización binaria compartida entre otros paquetes; runtime contiene el componente común de apagado que es utilizado por todos los elementos del sistema para el handleo de SIGTERM/SIGINT.

Las flechas apuntan desde el paquete que utiliza una funcionalidad hacia el paquete que la proporciona.

Diagrama de Despliegue

Despliegue mediante Docker Compose



El diagrama muestra una distribución física posible del sistema. El host cliente ejecuta la aplicación que trabaja con Parquet, envía datos y recibe resultados. Se comunica por TCP con los procesos gateway del host de acceso. Estos utilizan MOM para comunicarse por AMQP con RabbitMQ, ubicado en el host de mensajería y encargado de alojar exchanges y colas

Los hosts de cómputo ejecutan las instancias de workers, que también se comunican con RabbitMQ mediante AMQP y mantienen archivos por ejecución. El enlace HTTPS con Wikimedia corresponde a la obtención de imágenes para la Query 2. Las cajas interiores a cada host son los procesos y las líneas que unen los hosts, los enlaces de comunicación

La comunicación entre procesos permite separarlos físicamente mediante TCP y AMQP. El despliegue automatizado actual utiliza una red bridge de Docker Compose en un único host. La distribución entre varias computadoras requiere configurar conectividad, direcciones accesibles, montajes y ejecución en cada host; no está automatizada por el generador actual.

Resolución Distribuida de las 5 Queries

Esta sección mapea cómo la arquitectura reemplaza las funciones secuenciales de Pandas presentadas en la web referenciada de Kaggle.

- **Query 1 (Urls inglesas modificadas <= 2020):**
    - *Logica Kaggle:* Filtro de Dataframe df[(df['in_language'] == 'en') & (df['date_modified_year'] <= 2020)].

    - *Diseño:* El Worker extrae los campos. Si coinciden, emite el url y date_modified en un DTO. El Filtro Q1 lo transfiere directamente a escritura.
- **Query 2 (Biografías <= 2020):**
    - *Lógica Kaggle:* Carga JSON en memoria global, itera sobre diccionario buscando "biography" y cruza con fechas.
    - *Diseño:* El Worker ejecuta el parser JSON localmente en memoria pequeña para cada lote. Emite un DTO con name, date, image sólo si la condición es cierta. El Filtro Q2 agrupa la salida.
- **Query 3 (Palabras compartidas EN/FR):**
    - *Lógica Kaggle:* Intersección de sets english_words & french_words reteniendo todas las palabras en memoria a la vez.
    - *Diseño:* Los Workers etiquetan cada palabra resultante con el idioma de origen ("en", word) o ("fr", word). El Grouper Q4 los recibe y los descarta si ya existen en sus respectivos diccionarios locales (Sets Hash), optimizando espacio. Con el EOF de cierre, cruza localmente ambos diccionarios para obtener la intersección.
- **Query 4 (Top 20 palabras más comunes del abstract):**
    - *Lógica Kaggle:* Crea una colección collections.Counter() centralizada e itera todo el dataset usando NLTK local.
    - *Diseño:* El Worker limpia el texto y separa palabras, enviando mensajes unitarios (word, 1). La clave de ruteo en RabbitMQ aplica una función de hashing a la palabra. Cada Grouper Q3 acumula un Top 20 local en un diccionario. Al recibir EOF, el Sink junta estos Tops locales y ordena de manera global para emitir el Top 20 absoluto.
- **Query 5 (Longitud de artículos > 2020 respecto al promedio <= 2020):**
    - *Lógica Kaggle:* Dos pasadas en memoria. Primero calcula la media del dataframe <= 2020, luego aplica un lambda para contar palabras de la partición > 2020 y filtra.
    - *Diseño:* Se divide el flujo. Los *Workers NLP* derivan dos streams. Uno hacia el *Calculador Q5* con totales numéricos (length_words) para <= 2020 que consolida la media de forma matemática. El otro hacia el *Buffer Q5* de los datos > 2020 reteniendo solo URL y su cantidad calculada de palabras. Al concluir los primeros, la barrera se levanta inyectando el valor promedio en el Buffer; desencadenando la depuración y purga de aquellos que superen el índice estático provisto.

Lista de tareas y división

A partir del diseño planteado, las principales tareas a desarrollar que nos planteamos son las siguientes:

- El middleware de comunicación sobre RabbitMQ, que incluye el protocolo de propagación de EOF entre réplicas para el correcto cierre y para el procesamiento de queries que requieren del mismo para poder avanzar.
- El cliente y el gateway, comunicados por TCP mediante un protocolo propio.
- Lógica compartida: corresponde a la primera parte del sistema cuando se realizan los primeros filtros y luego el ruteo de la información hacia sus respectivas queries.
- Módulos de procesamiento: las cinco consultas con sus nodos asociados (filtros, descarga de imágenes, intersección de palabras, frecuencia, Top N, cálculo de promedio y comparación).
- Despliegue del sistema mediante docker-compose, con la cantidad de réplicas de cada tipo de worker configurable a través de variables de entorno.
- Graceful shutdown ante la señal SIGTERM para todos los procesos.

Para llevarlas adelante planteamos la siguiente división de trabajo:

Integrante A	Middleware + EOF	Q4	SIGTERM	
Integrante B	Gateway + Cliente	Q2 + lógica imágenes	Q3	Buffer de testeo e integración
Integrante C	Docker + lógica compartida	Q1	Q5	Buffer de testeo e integración

Este reparto busca lo siguiente

- **A:** implementa el protocolo de EOF y después lo usa en Q4, ya que depende de él, así el que conoce mejor el EOF es quien lo aplica en el Top N.
- **B:** construye el gateway, ya que es independiente en primera instancia y luego avanza con Q2 y Q3.
- **C:** arma la base común, que alimenta a Q1, Q3, Q4 y Q5, y después cierra Q1 y Q5. Q5 siendo la query de lógica más compleja.